

 No description has been provided for this image

Introduction to Pandas

pandas is a library with high-level data structures and manipulation tools:

DataFrame Object

A Pandas DataFrame is a two-dimensional size-mutable, potentially heterogeneous tabular data structure that holds **relational data**.

Data is aligned in a tabular fashion with labeled rows and columns like a spreadsheet or SQL table, or a dict of Series objects. Essentially, a Pandas DataFrame consists of three components, the data, rows, and columns.

Key takeaways:

- Represents a tabular, spreadsheet-like data structure
- Ordered collection of columns
- Each column can be a different value type (numeric, string, boolean, etc.)
- holds relational data

 No description has been provided for this image

Relational Data

Pandas dataframes/tables typically contain relational data. Relational data is data that captures associations or relationships between data points. This is often expressed as a table with columns indicating the quantities related. For example, "First names" are associated with "Last Names".

This is a table with two columns, one column for First Name and one column for Last Name. For the pedantic, a "relation" is a table with no duplicate entries.

 No description has been provided for this image

This is a table with two columns, one column for First Name and one column for Last Name.

For the pedantic, a "relation" is a table with no duplicate entries. We might, for example, have two John Smiths. We can try to eliminate this collision problem by introducing an **Index**.

 No description has been provided for this image

A brief note on row vs index In this notebook and lecture we will use the terms row number and index. For many contexts these terms are interchangeable, however due to pandas nomenclature around different ways of slicing and dicing data, we will have distinct meanings for these

row number: the number of the row starting with row 0 index: label given to a specific row



No description has been provided for this image

In the above table we have added an "Index". The `DataFrame` object in Python is a representation of a table with these components. It is composed of rows, each with an index, and labeled columns.

In a general `DataFrame`, we might have many different relations captured in the same table. For example, a `DataFrame` with student data from a school might look something like this:  No description has been provided for this image

Data Representation

When thinking about data analysis, note that the above table already gives us something to think about regarding how choices of data representation affect conclusions. What if someone only has one name? What if they have a name that isn't easily represented as "First name/Last name"? What if they come from a culture that keeps track of multiple names?

Note there is a Gender column. What if the person who constructed this data had created this column as `isFemale` ?

When interacting with tables and `DataFrames`, it is important to keep these issues in mind. Structural choices about data can and will affect conclusions. Whenever you make a `DataFrame` or use one someone else has constructed for you, you are making or dealing with these kinds of choices.

There are standard operations related to questions you might have about the students in the above `DataFrame`.

- Which students took Physics?
- Which students got an A in any course? Which Students from the University of Washington got an A or B in either Physics or History?

The `DataFrame` object has operations that allow these kinds of questions to be answered in a computationally efficient manner. These are covered in this tutorial.

Many different relations

If we only needed to subselect from one table, we wouldn't really need something like `pandas` and its `DataFrame` (though it is helpful for this!). The real power of the `DataFrame` object appears when we have multiple relations, i.e. multiple `DataFrame`s and we wish to combine them in some way.

For example we might have `DataFrame`s that represent student Grades, or Professors, or Schools.



No description has been provided for this image

or a `DataFrame` with Courses offered by Departments in different schools:



No description has been provided for this image

or a `DataFrame` of Professors and the Courses they teach



No description has been provided for this image

The main purpose of `pandas` and the `DataFrame` object is to allow us to ask questions across multiple sets of relations.

- What is the average score of students of course Y from professor X (who may have taught at different institutions....)?
- What is the average number of students at school X from State Y?
- What is the distribution of grades from students in Biology whose home town is Y?

`DataFrames` have powerful tools like `merge` to combine information from multiple `DataFrame`s that allow you to ask these kinds of questions quickly and easily.

Let's get started!

Why Use Pandas

Annotated Data is Powerful!

Because `pandas` combines data labels with values it facilitates easy:

- dataset exploration
- dataset visualization
- basic statistical analysis

Data manipulation is easy!

`Pandas` takes the functionality of slicing & dicing data and layers in many other helpful features that help with:

- cleaning data
 - Column headers are descriptive not numerical.
 - Columns hold single variables.
 - Variables are stored in either rows or columns, not both.
 - handling missing or invalid values
- data wrangling
 - getting the data into a structure that facilitates your analysis
- easily loading and saving data
 - load many formats of data and save in many formats

High Level Data manipulation

Pandas supports vectorized mathematical operations which optimizes computation performance and execution speed. Additionally the 2D labeled tabular structure of a pandas dataframe allows other high level manipulations such as:

- data grouping aggregation
- table manipulations (transforming rows to columns &/or columns to rows)
- Merging or Joining multiple dataframes/tables

Documentation & Resources

This introduction will only just scratch the surface of Pandas functionality. For more information, check out the [full documentation](#)

This [cheat-sheet](#) is also highly recommended to print out and keep handy as a resource.

Or check out the '[10 minutes to Pandas](#)' tutorial here (note: title may mischaracterize time investment).

Table of Contents & Jumlinks

Introduction

- Intro to Pandas and Dataframes
- Intro Relational Data
- why use pandas
- resources & documentation

Imports

Data Loading

Explore the dataset & basic functionality

- view the dataframe, how many rows & columns
- view specifics about columns
- get data types and unique entries
- descriptive statistics

Data Selection, slicing & dicing

- select data by labels/names
- select data by position
- "views" vs copies of the data
- conditional selection
- multiple condition selection

Table & Data Manipulation

- add/create new columns
- drop columns
- sort values

Table transformation, aggregation & plotting

- wide vs long tables
- melt (transform wide to long df)
- pivot (transform long to wide df)
- cross tab (aggregate counts)
- pivot_table (flexible aggregation)
- groupby & agg

Create new tables, join tables, save tables

- join & merge tables
- create dataframe from scratch
- saving dataframes

Import Packages

Library imports

Here we'll load in the libraries we'll use to shape and explore the data

```
In [1]: import os  
import numpy as np
```

```
In [2]: # import `pandas` and give it a short name `pd` since we will type it very  
import pandas as pd
```

Load dataset

Our first step is loading in the data from a file

Pandas has great [tools for automatically interpreting data from many sources](#)

- [pd.read_csv](#)
- [pd.read_feather](#)
- [pd.read_excel](#)
- [pd.read_pickle](#)
- [pd.read_json](#)

we will use `pd.read_feather`

```
In [3]: filepath = os.path.join('../support_files', 'datasets', 'messy_superstore.csv')
df = pd.read_feather(filepath)

# ignore this line of code for now- it will be explained a bit later
df.set_index('Row ID', inplace = True)
```

Explore the dataset

Internal links

basic data exploration

- [view dataframe "preview"](#)
- [view rows from beginning or end of dataframe \(df.head, df.tail\)](#)
- [get dataframe shape and length\(len, np.shape\)](#)
- [list all columns\(.columns\)](#)
- [get column data types\(.dtypes\)](#)

basic data visualization

descriptive statistics

- [get table level descriptive statistics](#)

Bonus Material (found at end of notebook)

[view the dataframe](#)

simply calling the dataframe ('df' or whatever you've named the dataframe) gives a preview view of the dataframe/table

- shows the first 5 and last 5 rows of data
- shows the first 10 and last 10 columns of data

In [4]: `df`

Out[4]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
IN-2014-23218	IN-2014-75456	Consumer	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-10004
IN-2014-24599	IN-2014-29767	Home Office	FURNITURE	NaN	BOOKCASES	Ikea Library with Doors, Mobile	FUR-10005
IN-2014-24597	IN-2014-29767	Home Office	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-10004
IN-2014-27993	IN-2014-20415	Home Office	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Pine	FUR-10002
IN-2014-28967	IN-2014-47337	Corporate	FURNITURE	NaN	CHAIRS	Hon Rocking Chair, Red	FUR-10003
...	...	...	...	...	...	...	...
ZA-2014-49187	ZA-2014-9750	Corporate	TECHNOLOGY	NaN	ACCESSORIES	Memorex Router, USB	TEC-10001
ZI-2014-42069	ZI-2014-7610	Corporate	TECHNOLOGY	NaN	MACHINES	StarTech Phone, Red	TEC-10000
ZI-2014-43712	ZI-2014-5970	Home Office	TECHNOLOGY	NaN	ACCESSORIES	Belkin Router, USB	TEC-10003
ZI-2014-48372	ZI-2014-9550	Consumer	TECHNOLOGY	NaN	MACHINES	Konica Receipt Printer, Red	TEC-10006
ZI-2014-48014	ZI-2014-3570	Consumer	TECHNOLOGY	NaN	MACHINES	Okidata Calculator, Red	TEC-10007

17531 rows x 34 columns

View the first or last n rows

`.head()`: shows the first n rows

`.tail()`: shows the last n rows

- `df.head()` and `df.tail()` shows 5 rows of data by default
- adding a number `df.head(n)` adjusts the number of rows shown

```
In [5]: # show the first 8 rows  
df.head(8)
```

Out[5]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
IN-2014-23218	IN-2014-75456	Consumer	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-FL1000406
IN-2014-24599	IN-2014-29767	Home Office	FURNITURE	NaN	BOOKCASES	Ikea Library with Doors, Mobile	FUR-BC1000125
IN-2014-24597	IN-2014-29767	Home Office	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-FL1000406
IN-2014-27993	IN-2014-20415	Home Office	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Pine	FUR-BC1000220
IN-2014-28967	IN-2014-47337	Corporate	FURNITURE	NaN	CHAIRS	Hon Rocking Chair, Red	FUR-CH1000396
AG-2014-50986	AG-2014-2760	Consumer	FURNITURE	NaN	FURNISHINGS	Deflect-O Light Bulb, Ergonomic	FUR-DE1000286
AG-2014-50983	AG-2014-2760	Consumer	FURNITURE	NaN	CHAIRS	Novimex Rocking Chair, Black	FUR-NO1000245
AG-2014-49384	AG-2014-2040	Consumer	FURNITURE	NaN	FURNISHINGS	Rubbermaid Frame, Durable	FUR-RUE1000300

8 rows x 34 columns

In [6]:

```
# show the last 7 rows
df.tail(7)
```

Out[6]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
ZA-2014-50068	ZA-2014-6660	Home Office	TECHNOLOGY	NaN	COPIERS	Brother Fax and Copier, High-Speed	TE BR 100034
ZA-2014-50069	ZA-2014-6660	Home Office	TECHNOLOGY	NaN	COPIERS	Hewlett Fax Machine, High-Speed	TE HE 100023
ZA-2014-49187	ZA-2014-9750	Corporate	TECHNOLOGY	NaN	ACCESSORIES	Memorex Router, USB	TE ME 100022
ZI-2014-42069	ZI-2014-7610	Corporate	TECHNOLOGY	NaN	MACHINES	StarTech Phone, Red	TEC-S1 100006
ZI-2014-43712	ZI-2014-5970	Home Office	TECHNOLOGY	NaN	ACCESSORIES	Belkin Router, USB	TEC-BI 100039
ZI-2014-48372	ZI-2014-9550	Consumer	TECHNOLOGY	NaN	MACHINES	Konica Receipt Printer, Red	TE KC 100037
ZI-2014-48014	ZI-2014-3570	Consumer	TECHNOLOGY	NaN	MACHINES	Okidata Calculator, Red	TEC-O 100014

7 rows x 34 columns

Get the shape and length of the dataframe

because pandas is built off of numpy, many familiar functions/methods work with DataFrames

- **numpys shape function** can give the dimensions of a dataframe
 - `df.shape` - returns (rows, columns)
- **len()** - the built in python function will return the number of rows in a dataframe
 - `len(df)`

```
In [7]: df.shape
```

```
Out[7]: (17531, 34)
```

```
In [8]: len(df)
```

```
Out[8]: 17531
```

List Columns

`.columns` provides a list of the column labels for a dataframe

```
df.columns
```

```
In [9]: # lets try it out  
df.columns
```

```
Out[9]: Index(['Order ID', 'Segment', 'Category', 'Category (OLD)', 'Sub-Categor  
y',  
              'Product Name', 'Product ID', 'Country', 'Market', 'Region', 'Quant  
ity',  
              'Discount', 'Profit', 'Customer ID', 'Customer Name', 'Order Priori  
ty',  
              'Postal Code', 'Ship Mode', 'Shipping Cost', '10/1/2014', '7/1/201  
4',  
              '11/1/2014', '9/1/2014', '1/1/2014', '12/1/2014', '8/1/2014',  
              '5/1/2014', '3/1/2014', '4/1/2014', '2/1/2014', '6/1/2014', 'City',  
              'State', 'manufacturers'],  
             dtype='object')
```

Show data from a single column

Like retrieving a value from a dictionary, we can get the data for a single column by indexing with the column's name:

```
In [10... df['Country']
```

```
Out[10]: Row ID
IN-2014-23218    Afghanistan
IN-2014-24599    Afghanistan
IN-2014-24597    Afghanistan
IN-2014-27993    Afghanistan
IN-2014-28967    Afghanistan
...
ZA-2014-49187    Zambia
ZI-2014-42069    Zimbabwe
ZI-2014-43712    Zimbabwe
ZI-2014-48372    Zimbabwe
ZI-2014-48014    Zimbabwe
Name: Country, Length: 17531, dtype: object
```

Get column or element data types

- **.dtypes** : lists the data type for all columns in a dataframe
 - `df.dtypes`
- **type()** allows inspection of data type for a specific element (i.e. specific row & column)
 - This can be helpful if the .dtypes function returns "object"
 - `type(df['column'][row number])`

```
In [11... df.dtypes
```

```
Out[11]: Order ID      object
        Segment      object
        Category      object
        Category (OLD) float64
        Sub-Category  object
        Product Name   object
        Product ID     object
        Country        object
        Market         object
        Region         object
        Quantity       float64
        Discount       float64
        Profit         float64
        Customer ID    object
        Customer Name   object
        Order Priority  object
        Postal Code    float64
        Ship Mode      object
        Shipping Cost   float64
        10/1/2014      float64
        7/1/2014       float64
        11/1/2014      float64
        9/1/2014       float64
        1/1/2014       float64
        12/1/2014      float64
        8/1/2014       float64
        5/1/2014       float64
        3/1/2014       float64
        4/1/2014       float64
        2/1/2014       float64
        6/1/2014       float64
        City           object
        State          object
        manufacturers  object
        dtype: object
```

```
In [12... # use the built in type() function to get the first row of the "Category" column
type(df["Category"]['ZA-2014-50068'])
```

```
Out[12]: str
```

get unique entries for a column

`.unique`

`df['column_name'].unique()` returns an array of all unique entries

```
In [13... # get all unique entries for the Country column
df['Country'].unique()
```

```
Out[13]: array(['Afghanistan', 'Algeria', 'Angola', 'Argentina', 'Australia',
               'Austria', 'Azerbaijan', 'Bangladesh', 'Barbados', 'Belarus',
               'Belgium', 'Bolivia', 'Brazil', 'Bulgaria', 'Cambodia', 'Cameroon',
               'Canada', 'Chile', 'China', 'Colombia', 'Cote d'Ivoire', 'Croatia',
               'Cuba', 'Czech Republic', 'Democratic Republic of the Congo',
               'Denmark', 'Dominican Republic', 'Ecuador', 'Egypt', 'El Salvador',
               'Estonia', 'Finland', 'France', 'Gabon', 'Georgia', 'Germany',
               'Ghana', 'Guatemala', 'Haiti', 'Honduras', 'Hungary', 'India',
               'Indonesia', 'Iran', 'Iraq', 'Ireland', 'Israel', 'Italy',
               'Jamaica', 'Japan', 'Jordan', 'Kazakhstan', 'Kenya', 'Kyrgyzstan',
               'Lebanon', 'Liberia', 'Libya', 'Lithuania', 'Macedonia',
               'Madagascar', 'Malaysia', 'Mali', 'Martinique', 'Mexico',
               'Moldova', 'Mongolia', 'Montenegro', 'Morocco', 'Mozambique',
               'Myanmar (Burma)', 'Nepal', 'Netherlands', 'New Zealand',
               'Nicaragua', 'Niger', 'Nigeria', 'Norway', 'Pakistan', 'Panama',
               'Paraguay', 'Peru', 'Philippines', 'Poland', 'Portugal',
               'Republic of the Congo', 'Romania', 'Russia', 'Rwanda',
               'Saudi Arabia', 'Senegal', 'Singapore', 'Slovenia', 'Somalia',
               'South Africa', 'South Korea', 'Spain', 'Sudan', 'Sweden',
               'Switzerland', 'Syria', 'Tanzania', 'Thailand', 'Togo', 'Tunisia',
               'Turkey', 'Turkmenistan', 'Uganda', 'Ukraine', 'United Kingdom',
               'United States', 'Uruguay', 'Uzbekistan', 'Venezuela', 'Vietnam',
               'Yemen', 'Zambia', 'Zimbabwe', 'Albania', 'Benin',
               'Bosnia and Herzegovina', 'Central African Republic', 'Ethiopia',
               'Guinea', 'Guinea-Bissau', 'Hong Kong', 'Mauritania', 'Namibia',
               'Papua New Guinea', 'Qatar', 'Sierra Leone', 'Slovakia',
               'Sri Lanka', 'Taiwan', 'Tajikistan', 'Trinidad and Tobago',
               'United Arab Emirates', 'South Sudan', 'Swaziland'], dtype=object)
```

STUDENT EXERCISES:

1. What are the unique values for the Market column?
2. Identify the data type for the Product ID column

```
In [14... # Answer 1:
df['Market'].unique()
```

```
Out[14]: array(['APAC', 'Africa', 'LATAM', 'EU', 'EMEA', 'Canada', 'US'],
               dtype=object)
```

```
In [15... # Answer 2:
type(df["Product ID"])
```

```
Out[15]: pandas.core.series.Series
```

Plotting with Pandas!

Pandas has a set of plotting tools built into the library. You can quickly create visualizations(especially with grouped and aggregated data which we will get to later) using pandas built in plotting functions such as:

- `.plot`

- example:

```
df.plot(x = 'xcolumn', y= 'ycolumn', kind = "box", xlabel= 'x value units',  
        ylabel = 'y value units')
```

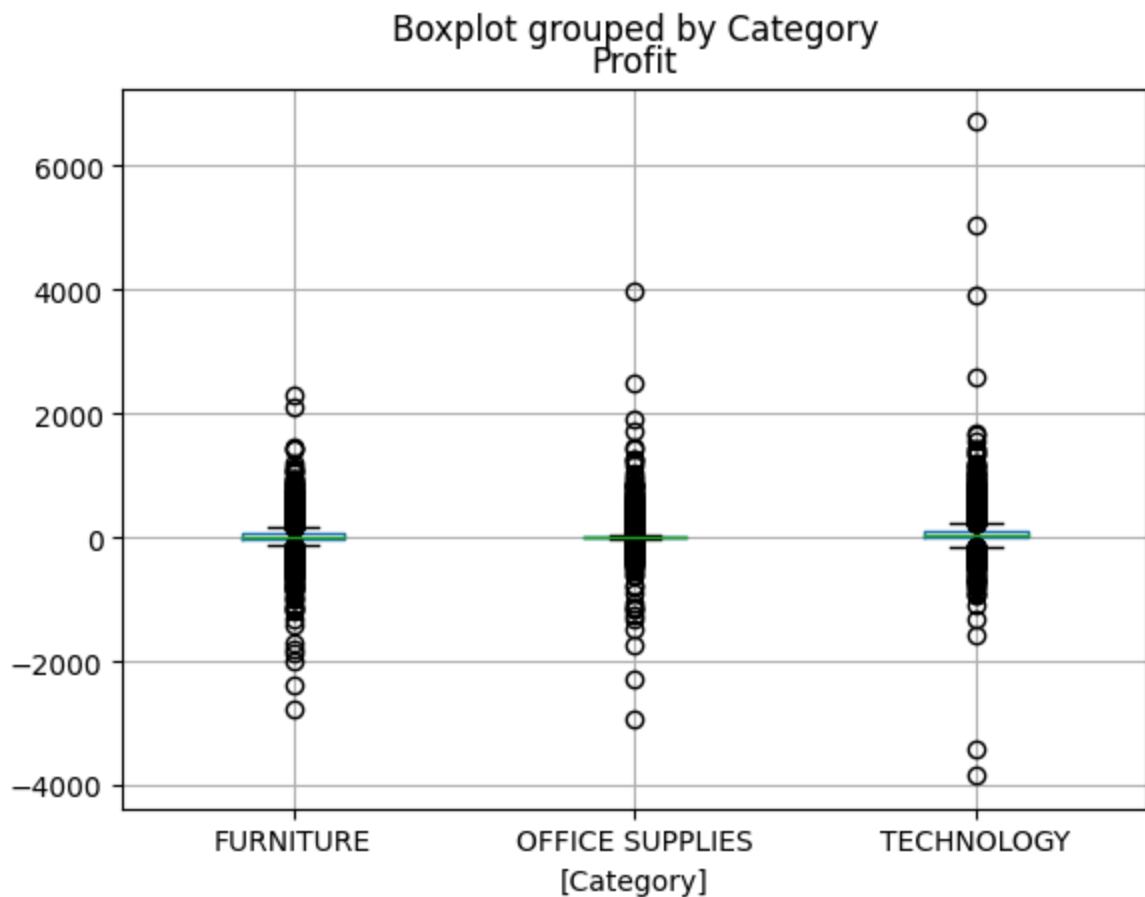
- `.boxplot`

NOTE! Pandas works especially well with the the seaborn plotting package. We won't cover this now because there's a whole data visualization tutorial later!

```
In [16...] # Lets create a boxplot of profit by category
```

```
df.boxplot(column=['Profit'], by=['Category'])
```

```
Out[16]: <Axes: title={'center': 'Profit'}, xlabel='[Category] '>
```



get descriptive statistics

Descriptive statistic are summary statistics that quantitatively describes or summarizes features from a collection of information or dataset. This typically included things like sample size, measures of central tendency (mean, median,

mode), measures of variability or dispersion (standard deviation, min, max, kurtosis, skewness)

`.describe()` will return descriptive statistics for all quantitative columns of a dataframe

 No description has been provided for this image

```
df.describe()
```

For each numerical column the following descriptive statistics are provided:

- count
- mean
- standard deviation
- minimum
- 25, 50 & 75th percentiles
- max

In [17... `df.describe()`

Out[17]:

	Category (OLD)	Quantity	Discount	Profit	Postal Code	Ship Mode
count	0.0	17531.000000	17531.000000	17531.000000	3321.000000	17531.000000
mean	NaN	3.457989	0.143291	28.758540	56192.587775	26.260000
std	NaN	2.290856	0.211830	174.283412	31977.397359	56.540000
min	NaN	1.000000	0.000000	-3839.990400	1841.000000	0.000000
25%	NaN	2.000000	0.000000	0.000000	28110.000000	2.500000
50%	NaN	3.000000	0.000000	9.200000	60440.000000	7.750000
75%	NaN	5.000000	0.200000	36.808500	90032.000000	24.450000
max	NaN	14.000000	0.800000	6719.980800	99301.000000	867.690000

other built in summary statistic functions

Pandas also provides a large set of summary functions that can operate on different kinds of pandas objects (dataframe columns, Series, Groupby etc.)

- `.count()`
- `.sum()`
- `.min()`
- `.max()`
- `.mean()`
- `.median()`

- `.var()`
- `.std()`

These functions become especially useful in the next section where there might be some specific selection of data you want to analyze

```
In [18... # lets do a very quick demonstration of how this works
# here we get the mean discount for the entire discount column
df["Discount"].mean()
```

Out[18]: 0.14329119844846275

STUDENT EXERCISE:

What is the maximum value in the Shipping Cost column?

```
In [19... df['Shipping Cost'].max()
```

Out[19]: 867.69

Data Selection, Slicing & Dicing

Internal Links

basic data selection & filtering

- [column selection](#)
- ['view' of data vs a copy](#)
- [select data by labels/names](#)
- [a note on row number vs index](#)
- [select data by integer Index/position](#)

advanced data selection

- [select data for a given condition or threshold](#)
- [return subset of data for a given condition or threshold](#)
- [select data based on multiple conditions or thresholds](#)

Bonus Material (found at end of notebook)

- [multiple condition options using a list](#)

Column Selection

column selection

 No description has been provided for this image
to view a single single column

- `df['column_name']` (returns a pandas Series object, which is similar to a 1D array or list)

~ OR ~

- `df[['column_name']]` (returns a DataFrame)

multiple columns:

- `df[['column_1', 'columns_2']]`

```
In [20... # get the Category column as a pd.series
```

```
df["Category"]
```

```
Out[20]: Row ID
IN-2014-23218    FURNITURE
IN-2014-24599    FURNITURE
IN-2014-24597    FURNITURE
IN-2014-27993    FURNITURE
IN-2014-28967    FURNITURE
...
ZA-2014-49187    TECHNOLOGY
ZI-2014-42069    TECHNOLOGY
ZI-2014-43712    TECHNOLOGY
ZI-2014-48372    TECHNOLOGY
ZI-2014-48014    TECHNOLOGY
Name: Category, Length: 17531, dtype: object
```

```
In [21... # get the Category and Customer ID columns
```

```
df[["Category", "Customer ID"]]
```

Out[21]:

	Category	Customer ID
Row ID		
IN-2014-23218	FURNITURE	AA-10375
IN-2014-24599	FURNITURE	CA-12055
IN-2014-24597	FURNITURE	CA-12055
IN-2014-27993	FURNITURE	GM-14455
IN-2014-28967	FURNITURE	VB-21745
...	...	...
ZA-2014-49187	TECHNOLOGY	TS-11205
ZI-2014-42069	TECHNOLOGY	BS-1380
ZI-2014-43712	TECHNOLOGY	JB-6045
ZI-2014-48372	TECHNOLOGY	JC-5775
ZI-2014-48014	TECHNOLOGY	NG-8430

17531 rows × 2 columns

Views vs copies

We often want to work with only a subset of a dataframe. For that purpose, we can select only those rows or columns that we need and leave the rest.

Building off of our knowledge of views vs copies that we learned with our numpy tutorial, when we subset an array the result is not always a new array; sometimes what numpy returns is a view of the data in the original array. Since pandas Series and DataFrames are backed by numpy arrays, it will probably come as no surprise that something similar sometimes happens in pandas. Unfortunately, while this behavior is relatively straightforward in numpy, in pandas there's just no getting around the fact that it's a hot mess.

The View/Copy Headache in pandas: In numpy, the rules for when you get views and when you don't are a little complicated, but they are consistent: certain behaviors (like simple indexing) will always return a view, and others (fancy indexing) will never return a view.

But in pandas, whether you get a view or not—and whether changes made to a view will propagate back to the original DataFrame—depends on the structure and data types in the original DataFrame

 No description has been provided for this image

When to create a copy using `.copy()` Creating copies of dataframes can be especially useful when doing exploratory analysis on a dataframe where you want to keep the integrity of the original dataframe in case you make a mistake.

`new_df = df.copy()` makes a copy (by creating a new object) of this object's indices and data. By default modifications to the data or indices of the copy will not be reflected in the original object but see the documentation and the parameter `deep` for more information

Note: Like all other variables, try to keep your dataframe naming descriptive and intuitive to read!(For example "new_df" would be a bad name)



No description has been provided for this image



No description has been provided for this image

Select rows or columns using labels

`.loc()` allows access to a group of rows and columns by label(s) or a boolean array.

select rows:

- `df.loc['index']`
 - hint: similar to column selection described above, for row selection using `df.loc[ ]` returns a series where `df.loc[[ ]]` returns a dataframe

select row and column:

- `df.loc['index', 'column']`

```
In [22... # get rows where the index is IN-2014-28967
df.loc["IN-2014-28967"]
```

```

Out[22]: Order ID                IN-2014-47337
        Segment                Corporate
        Category                FURNITURE
        Category (OLD)         NaN
        Sub-Category           CHAIRS
        Product Name           Hon Rocking Chair, Red
        Product ID             FUR-CH-10003965
        Country                Afghanistan
        Market                 APAC
        Region                 Central Asia
        Quantity               7.0
        Discount               0.0
        Profit                 356.58
        Customer ID            VB-21745
        Customer Name          Victoria Brennan
        Order Priority          High
        Postal Code            NaN
        Ship Mode              Standard Class
        Shipping Cost          106.41
        10/1/2014              NaN
        7/1/2014               NaN
        11/1/2014              NaN
        9/1/2014               NaN
        1/1/2014               NaN
        12/1/2014              914.34
        8/1/2014               NaN
        5/1/2014               NaN
        3/1/2014               NaN
        4/1/2014               NaN
        2/1/2014               NaN
        6/1/2014               NaN
        City                   Kabul
        State                   Kabul
        manufacturers          [ACME Co, Buy n Large, Dunder Mifflin, LexCorp...
        Name: IN-2014-28967, dtype: object

```

```

In [23]: # get index:ZI-2014-48372 and column'Country'
df.loc["ZI-2014-48372", 'Country']

```

```

Out[23]: 'Zimbabwe'

```

Select rows and columns by position

`.iloc()` allows positional selection ie row number, column number.

(Note that the same slicing notation that we saw with numpy arrays works with `iloc`.)

Example: `df.iloc([row(s) number, column(s) number])`

Examples:

- select all rows and all columns

- `df.iloc[:,:]`
- select first 5 rows and all columns
 - `df.iloc[0:4, :]`
- select all rows and last 5 columns
 - `.iloc[:, -5:]`

In [24... *# get the first 9 rows and all columns*
`df.iloc[:9,:]`

Out[24]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
IN-2014-23218	IN-2014-75456	Consumer	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-FL1000406
IN-2014-24599	IN-2014-29767	Home Office	FURNITURE	NaN	BOOKCASES	Ikea Library with Doors, Mobile	FUR-BC1000125
IN-2014-24597	IN-2014-29767	Home Office	FURNITURE	NaN	FURNISHINGS	Rubbermaid Door Stop, Ergonomic	FUR-FL1000406
IN-2014-27993	IN-2014-20415	Home Office	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Pine	FUR-BC1000220
IN-2014-28967	IN-2014-47337	Corporate	FURNITURE	NaN	CHAIRS	Hon Rocking Chair, Red	FUR-CH1000396
AG-2014-50986	AG-2014-2760	Consumer	FURNITURE	NaN	FURNISHINGS	Deflect-O Light Bulb, Ergonomic	FUR-DE1000286
AG-2014-50983	AG-2014-2760	Consumer	FURNITURE	NaN	CHAIRS	Novimex Rocking Chair, Black	FUR-NO1000245
AG-2014-49384	AG-2014-2040	Consumer	FURNITURE	NaN	FURNISHINGS	Rubbermaid Frame, Durable	FUR-RUE1000300
AG-2014-47438	AG-2014-2600	Home Office	FURNITURE	NaN	BOOKCASES	Ikea 3-Shelf Cabinet, Pine	FUR-IKE1000364

9 rows × 34 columns

In [25... `# get all rows and last 4 columns`
`df.iloc[:,-4:]`

Out[25]:

	6/1/2014	City	State	manufacturers
Row ID				
IN-2014-23218	NaN	Kabul	Kabul	[Dunder Mifflin, Globex Corp, Hudsucker Indust...
IN-2014-24599	NaN	Herat	Hirat	[ACME Co, Buy n Large, Dunder Mifflin, Globex ...
IN-2014-24597	NaN	Herat	Hirat	[Dunder Mifflin, LexCorp, Olivander Crafts, Ro...
IN-2014-27993	NaN	Kabul	Kabul	[Dunder Mifflin, Olivander Crafts]
IN-2014-28967	NaN	Kabul	Kabul	[ACME Co, Buy n Large, Dunder Mifflin, LexCorp...
...	...	...	...	...
ZA-2014-49187	NaN	Ndola	Copperbelt	[ACME Co, Royco Waystar, Umbrella Corporation]
ZI-2014-42069	NaN	Bulawayo	Bulawayo	[Dunder Mifflin]
ZI-2014-43712	77.688	Bulawayo	Bulawayo	[ACME Co, Buy n Large, Hudsucker Industries]
ZI-2014-48372	NaN	Bulawayo	Bulawayo	[ACME Co, Wayne Enterprises]
ZI-2014-48014	NaN	Harare	Harare	[Dunder Mifflin, Olivander Crafts, Royco Wayst...

17531 rows × 4 columns

Select data given a specific threshold or condition

You can conditionally select data using `.loc[ ]` or `.query()` or directly on the dataframe. Each of these utilize boolean masking. `.loc`

- `df.loc[df['column'] > 7]`
- `df.loc[df['column'] == 'string']`
 - Pros: explicit, can be easier to read for those who already know python
 - cons: more verbose than query

`.query()` : This method uses boolean expressions and may be easier & more intuitive for those who know sql or other database languages.

Examples:

- `df.query('column > 7')`
- `df.query('column == string')`
 - Pros: can be easier for those who know sql or other database languages, less verbose
 - cons: can be difficult for those that don't know sql, doesn't handle column names that contain spaces very well

In [26... *# use .loc to select where Profit > 500*
`df.loc[df['Profit'] > 500]`

Out[26]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Price
IN-2014-27993	IN-2014-20415	Home Office	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Pine	FU 100
AO-2014-44396	AO-2014-2270	Consumer	FURNITURE	NaN	TABLES	Chromcraft Round Table, Adjustable Height	100
IN-2014-24624	IN-2014-23880	Consumer	FURNITURE	NaN	BOOKCASES	Sauder Classic Bookcase, Traditional	FU 100
IN-2014-31153	IN-2014-80286	Consumer	FURNITURE	NaN	CHAIRS	Harbour Creations Executive Leather Armchair, ...	FU 100
ID-2014-21496	ID-2014-35892	Consumer	FURNITURE	NaN	CHAIRS	SAFCO Executive Leather Armchair, Set of Two	FU 100
...	...	...	...	...	...	...	...
CA-2014-33921	CA-2014-127180	Home Office	TECHNOLOGY	NaN	PHONES	Polycom CX600 IP Phone VoIP phone	TE 100
CA-2014-33920	CA-2014-127180	Home Office	TECHNOLOGY	NaN	COPIERS	Canon imageCLASS 2200 Advanced Copier	TE 100
CA-2014-36178	CA-2014-143567	Corporate	TECHNOLOGY	NaN	ACCESSORIES	Logitech diNovo Edge Keyboard	TE 100
CA-2014-37637	CA-2014-143112	Corporate	TECHNOLOGY	NaN	MACHINES	3D Systems Cube Printer, 2nd Generation, Magenta	TE 100
CA-2014-38429	CA-2014-141439	Home Office	TECHNOLOGY	NaN	PHONES	Samsung Galaxy S4 Mini	TE 100

	Order ID	Segment	Category	Category (OLD)	Sub- Category	Product Name	Pr
Row ID							

231 rows x 34 columns

Return a subset of data given a specific threshold or condition

You can also return specific columns after doing conditional selection data using `.loc`.

single return column:

- `df.loc[df['selection_column'] > 7, 'return_column']`

multiple return columns (provide columns as a list):

- `df.loc[df['selection_column'] > 7, ['return_column1', 'return_column2']]`

```
In [27... # get the Category and Sub-Category where profit is above 500
df.loc[df["Profit"] > 500, ["Category", "Sub-Category"]]
```

Out[27]:

	Category	Sub-Category
Row ID		
IN-2014-27993	FURNITURE	BOOKCASES
AO-2014-44396	FURNITURE	TABLES
IN-2014-24624	FURNITURE	BOOKCASES
IN-2014-31153	FURNITURE	CHAIRS
ID-2014-21496	FURNITURE	CHAIRS
...	...	...
CA-2014-33921	TECHNOLOGY	PHONES
CA-2014-33920	TECHNOLOGY	COPIERS
CA-2014-36178	TECHNOLOGY	ACCESSORIES
CA-2014-37637	TECHNOLOGY	MACHINES
CA-2014-38429	TECHNOLOGY	PHONES

231 rows x 2 columns

Multiple condition selection

You may wish to select rows or columns where multiple conditions are met. You can combine conditions with `.loc` and like in numpy you will utilize `&` and `|`

example: `df.loc[(df['column1']=='string') & (df['column2'] > threshold)]`

```
In [28... # return all entries where Profit is greater than 900 and the Sub-Category
# the & means both conditions must be met

df.loc[(df['Sub-Category']=="BOOKCASES") & (df['Profit'] >900)]
```

Out[28]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
IN-2014-24624	IN-2014-23880	Consumer	FURNITURE	NaN	BOOKCASES	Sauder Classic Bookcase, Traditional	FUR-BO1000485
IN-2014-21263	IN-2014-56206	Consumer	FURNITURE	NaN	BOOKCASES	Sauder Classic Bookcase, Metal	FUR-BO1000147
IN-2014-24037	IN-2014-17930	Consumer	FURNITURE	NaN	BOOKCASES	Sauder Classic Bookcase, Metal	FUR-BO1000147
IN-2014-28056	IN-2014-66573	Consumer	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Mobile	FUR-BO1000466
MX-2014-4452	MX-2014-157077	Consumer	FURNITURE	NaN	BOOKCASES	Dania Classic Bookcase, Traditional	FUR-BO1000230
IN-2014-25795	IN-2014-76016	Corporate	FURNITURE	NaN	BOOKCASES	Sauder Classic Bookcase, Traditional	FUR-BO1000485
IT-2014-16653	IT-2014-4540740	Consumer	FURNITURE	NaN	BOOKCASES	Safco Classic Bookcase, Metal	FUR-BO1000499
ES-2014-12449	ES-2014-4957212	Consumer	FURNITURE	NaN	BOOKCASES	Bush Classic Bookcase, Pine	FUR-BO1000470
ZA-2014-42448	ZA-2014-7540	Consumer	FURNITURE	NaN	BOOKCASES	Ikea Library with Doors, Traditional	FUR-IKE1000289

9 rows x 34 columns

```
In [29... # return all entries where the Country is "Australia" or the Country is "Zambia"
# the | means at least one condition must be met

df.loc[(df['Country']=="Australia") | (df['Country']=="Zambia")]
```

Out[29]:

Row ID	Order ID	Segment	Category	Category (OLD)	Sub-Category	Product Name	Product ID
ID-2014-25801	ID-2014-42388	Consumer	FURNITURE	NaN	FURNISHINGS	Deflect-O Frame, Duo Pack	FUR-F1000470
IN-2014-21802	IN-2014-25644	Consumer	FURNITURE	NaN	BOOKCASES	Bush Corner Shelving, Metal	FUR-B1000420
IN-2014-22521	IN-2014-10090	Consumer	FURNITURE	NaN	CHAIRS	Harbour Creations Steel Folding Chair, Red	FUR-C1000120
IN-2014-26860	IN-2014-64396	Home Office	FURNITURE	NaN	FURNISHINGS	Deflect-O Stacking Tray, Black	FUR-F1000110
ID-2014-30742	ID-2014-84164	Home Office	FURNITURE	NaN	FURNISHINGS	Tenex Photo Frame, Duo Pack	FUR-F1000050
...	...	...	...	...	...	...	...
ZA-2014-41769	ZA-2014-1370	Consumer	TECHNOLOGY	NaN	COPIERS	Brother Personal Copier, Color	TE-BR1000390
ZA-2014-45397	ZA-2014-8330	Corporate	TECHNOLOGY	NaN	PHONES	Cisco Headset, VoIP	TEC-CI1000340
ZA-2014-50068	ZA-2014-6660	Home Office	TECHNOLOGY	NaN	COPIERS	Brother Fax and Copier, High-Speed	TE-BR1000340
ZA-2014-50069	ZA-2014-6660	Home Office	TECHNOLOGY	NaN	COPIERS	Hewlett Fax Machine, High-Speed	TE-HEV1000230
ZA-2014-	ZA-2014-	Corporate	TECHNOLOGY	NaN	ACCESSORIES	Memorex Router,	TE-MEI

	Order ID	Segment	Category	Category (OLD)	Sub- Category	Product Name	Produ
Row ID							
49187	9750					USB	100022

1023 rows × 34 columns

STUDENT EXERCISES:

1. Select Row ID "AG-2014-50983" and all columns
2. Get the mean profit when the Category is furniture
3. How many entries in the dataframe are there where "Profit" is negative and "Market" is 'EU'

In [30...

```
# Answer 1:
df.loc["AG-2014-50983"]
```



```

Out[30]: Order ID          AG-2014-2760
        Segment          Consumer
        Category          FURNITURE
        Category (OLD)    NaN
        Sub-Category      CHAIRS
        Product Name      Novimex Rocking Chair, Black
        Product ID        FUR-NOV-10002453
        Country           Algeria
        Market            Africa
        Region            Africa
        Quantity          4.0
        Discount          0.0
        Profit            61.92
        Customer ID       CL-2565
        Customer Name      Clay Ludtke
        Order Priority     High
        Postal Code        41244.0
        Ship Mode          Standard Class
        Shipping Cost      46.75
        10/1/2014         NaN
        7/1/2014          NaN
        11/1/2014         NaN
        9/1/2014          NaN
        1/1/2014          NaN
        12/1/2014         NaN
        8/1/2014          NaN
        5/1/2014          NaN
        3/1/2014          NaN
        4/1/2014          NaN
        2/1/2014          NaN
        6/1/2014          516.0
        City              Saida
        State              Saida
        manufacturers      [Dunder Mifflin, Umbrella Corporation]
        Name: AG-2014-50983, dtype: object

```

```

In [31... # Answer 2 with loc:
df.loc[df['Category']=='FURNITURE', 'Profit'].mean()

```

```

Out[31]: 26.68421359426352

```

```

In [32... # Answer 3
# Return the dataframe where "Profit" is negative and "Market" is 'EU'

# using .loc:
len(df.loc[(df['Profit'] < 0) & (df['Market'] == 'EU')])

# using .query:
len(df.query('Profit < 0 & Market == "EU"'))

```

```

Out[32]: 768

```

Internal links

- [adjust column type](#)
- [create new columns](#)
 - [.apply\(\)](#)
- [drop unnecessary or redundant columns](#)
- [drop nan, NULL entries](#)
- [sort the whole dataframe based on one column's values](#)

change column data type

After examining your dataframe you may find that there are certain columns whose data types do not match your assumptions or whose data types could inhibit analysis. For example, if you have a column that contains numbers, but the data type is string- you would not actually be able to do numeric operations on it. You can cast the data to a different datatype using [.astype\(\)](#).

to cast a column to a new data type: `df['column_1'] = df['column_1'].astype(new_type)`

remember some of the possible datatypes are:

- int
- float64
- bool
- string

```
In [33... # we know that in our dataframe quantity should be an integer
# but right now it's a float- lets change that using .astype()
# then we can check the type after

# change the type to an int
df['Quantity'] = df['Quantity'].astype(int)

# check the data type
df["Quantity"]
```

```
Out[33]: Row ID
IN-2014-23218    2
IN-2014-24599    2
IN-2014-24597    4
IN-2014-27993    5
IN-2014-28967    7
..
ZA-2014-49187    1
ZI-2014-42069    1
ZI-2014-43712    1
ZI-2014-48372    2
ZI-2014-48014    1
Name: Quantity, Length: 17531, dtype: int64
```

Add new columns

Frequently you'll want to add **new columns**. There are multiple ways to do this!

No description has been provided for this image to add already computed data: *

```
df['new_column_name'] = already_computed_data
```

You can also calculate new columns based on currently existing columns:

- `df['AB_SUM'] = df['A'] + df['B']`
- `df['Volume'] = df['Length'] * df['Height'] * df['Depth']`

note: newly created columns are always added to the end of the dataframe

```
In [34... # lets create a "Profit Per Unit" column by diving profit by quantity
df['Profit Per Unit'] = df['Profit'] / df['Quantity']

# now lets view the results
df[["Profit", "Quantity", "Profit Per Unit"]]
```

Out[34]:

	Profit	Quantity	Profit Per Unit
Row ID			
IN-2014-23218	39.840	2	19.920
IN-2014-24599	102.420	2	51.210
IN-2014-24597	79.680	4	19.920
IN-2014-27993	848.700	5	169.740
IN-2014-28967	356.580	7	50.940
...	...	...	...
ZA-2014-49187	19.710	1	19.710
ZI-2014-42069	-20.799	1	-20.799
ZI-2014-43712	-59.562	1	-59.562
ZI-2014-48372	-93.180	2	-46.590
ZI-2014-48014	-20.298	1	-20.298

17531 rows × 3 columns

.apply()

.apply() is a fairly powerful tool that can do many things. In this context we will use it to create new columns using functions from other packages or functions you've created.

- `df['new_column_name'] = df[column(s) of interest].apply(function)`
- `df['A_sqrt'] = df['A'].apply(np.sqrt)`
- `df['A_adjusted'] = df['A'].apply(your_function)`

In [35... *# lets practice by making a function that will fix the capitalization of the "Category" column and then applying it*

```
def capitalize_first_letter(string):
    all_lower = string.lower()
    cap_first = all_lower.capitalize()
    return cap_first

df['Category'] = df['Category'].apply(capitalize_first_letter)

# view the results
df[['Category']]
```

Out[35]:

Category	
Row ID	
IN-2014-23218	Furniture
IN-2014-24599	Furniture
IN-2014-24597	Furniture
IN-2014-27993	Furniture
IN-2014-28967	Furniture
...	...
ZA-2014-49187	Technology
ZI-2014-42069	Technology
ZI-2014-43712	Technology
ZI-2014-48372	Technology
ZI-2014-48014	Technology

17531 rows × 1 columns

STUDENT EXERCISES:

1. In these code examples, what do you think the `axis` argument is doing? What kind of output do you think will be returned? for each?

- `df[['A', 'B']].apply(np.mean, axis = 0)`
- `df[['A', 'B']].apply(np.mean, axis = 1)`

2. Which axis setting is the default? (hint- check the documentation)

Solution 1:

- 0 or 'index': apply function to each column.
- 1 or 'columns': apply function to each row.

```
In [36... # code to see the output differences
df[["Quantity", "Profit Per Unit"]].apply(np.sum, axis = 0)
```

```
Out[36]: Quantity          60622.00000
Profit Per Unit    144656.52516
dtype: float64
```

```
In [37... # code to see the output differences
df[["Quantity", "Profit Per Unit"]].apply(np.sum, axis = 1)
```

```
Out[37]: Row ID
IN-2014-23218      21.920
IN-2014-24599      53.210
IN-2014-24597      23.920
IN-2014-27993     174.740
IN-2014-28967      57.940
...
ZA-2014-49187      20.710
ZI-2014-42069     -19.799
ZI-2014-43712     -58.562
ZI-2014-48372     -44.590
ZI-2014-48014     -19.298
Length: 17531, dtype: float64
```

Solution 2:

0 is the default- meaning it will automatically apply to columns.

lambda functions

lambda is a built in python function that is often used with `.apply()` for pandas data manipulations.

Python Lambda Functions are anonymous function means that the function is without a name. As we already know that the `def` keyword is used to define a normal function in Python. Similarly, the `lambda` keyword is used to define an anonymous function in Python.

- syntactically restricted to a single expression
- Good for performing short operations/data manipulations
- generally reduce the readability of code
- `lambda input_data: data manipulation`
- `lambda string: string.lower().capitalize()`

Defined function:

```
def cube(num):
    return num * num * num
```

Lambda equivalent:

```
lambda num: num * num * num
```

STUDENT EXERCISES:

1. create a lambda function that adds 100 to an input value

```
In [38... # solution 1
plus_100 = lambda num: num + 100
```

using `.apply(lambda)`

`.apply(lambda)` is frequently used for row-wise operations, though there are several ways it can be utilized.

In this setting lambda is the function that will be used by `.apply`

- `data.apply(lambda input: manipulation)`

column-wise generic example:

- `df['text column'].apply(lambda string: string.lower().capitalize())`

row-wise generic example:

- `df.apply(lambda row: df['column_A']*df['column_B'], axis = 1)`

```
In [39... # lets use .apply(lambda) to create a new column that will count the number
# from the 'manufacturers' column array

df['manufacturers_count'] = df.apply(lambda row : len(row['manufacturers']),
```

STUDENT EXERCISES:

1. use `.apply(lambda )` to apply the `capitalize_first_letter` function to the 'Sub-Category' column

```
In [40... # solution 1
df['Sub-Category'] = df['Sub-Category'].apply(lambda string: capitalize_fi
```

Drop specific columns & rows using

You can easily drop rows or columns using `.drop()`

There are many reasons you may wish to drop columns from your dataframe. Here are a few examples:

- irrelevant to the analysis you are going to do

- out of date
- redundant information
- non-analyzable due to high rates of Nan or empty entries

drop columns:

- `df = df.drop(columns=['column1', 'columns2'])`
- `df = df.drop('column_name',axis=1)`

drop rows:

- `df = df.drop(index=('index1_string', 'index2_string'))`
- `df = df.drop([row, row])`

```
In [41]: # Lets drop the 'Category (OLD)' column

df.drop(columns=['Category (OLD)'], inplace=True)

# check the results in the dataframe columns list
df.columns

Out[41]: Index(['Order ID', 'Segment', 'Category', 'Sub-Category', 'Product Name',
               'Product ID', 'Country', 'Market', 'Region', 'Quantity', 'Discount',
               'Profit', 'Customer ID', 'Customer Name', 'Order Priority',
               'Postal Code', 'Ship Mode', 'Shipping Cost', '10/1/2014', '7/1/2014',
               '11/1/2014', '9/1/2014', '1/1/2014', '12/1/2014', '8/1/2014',
               '5/1/2014', '3/1/2014', '4/1/2014', '2/1/2014', '6/1/2014', 'City',
               'State', 'manufacturers', 'Profit Per Unit', 'manufacturers_count'],
              dtype='object')
```

Modifying data inplace

`inplace` is a parameter accepted by a number of pandas methods which affects the behaviour of how the method runs. Some examples of where you might commonly see this keyword are the methods(non-exhaustive):

- `.drop()`
- `.fillna()`
- `.replace()`
- `.rename()`

Using the `inplace=True` keyword in a pandas method changes the default behaviour such that the operation on the dataframe doesn't return anything, it instead 'modifies the underlying data'. It mutates the actual object which you apply it to.

remove rows with missing values

For columns of particular importance may wish to exclude rows that have missing values to do this we use `.dropna()`

example: `df.dropna(subset=['column'], inplace=True)`

In [42... *# we will also have an example of this in the next section!*

Sort the dataframe based on column values

`.sort_values()` to sort your dataframe by the the values of a particular column or columns.

```
df.sort_values(by=['column1', 'column2'], ascending = boolean,
inplace= boolean)
```

pro-tip: Sorting the values of your dataframe before plotting creates more interpretable plots.

In [43... *# Lets sort our USA_df by 'Segment', 'Category' and 'Sub-Category'*
we will leave the underlying data unchanged by setting inplace to False

```
df.sort_values(by = ['Segment', 'Category', 'Sub-Category'], inplace = False)
```

Out[43]:

Row ID	Order ID	Segment	Category	Sub-Category	Product Name	Product ID
AG-2014-46003	AG-2014-8600	Consumer	Furniture	Bookcases	Sauder Classic Bookcase, Pine	FUR-SAU-10002255
AO-2014-45199	AO-2014-9370	Consumer	Furniture	Bookcases	Bush Classic Bookcase, Pine	FUR-BUS-10001243
US-2014-6344	US-2014-165512	Consumer	Furniture	Bookcases	Bush Library with Doors, Metal	FUR-BO-10002968
US-2014-1158	US-2014-107664	Consumer	Furniture	Bookcases	Ikea Floating Shelf Set, Traditional	FUR-BO-10001585
IN-2014-21802	IN-2014-25644	Consumer	Furniture	Bookcases	Bush Corner Shelving, Metal	FUR-BO-10004230
...	...	...	...	...	...	...
US-2014-40156	US-2014-114356	Home Office	Technology	Phones	Plantronics Encore H101 Dual Earpieces Headset	TEC-PH-10003171
US-2014-33227	US-2014-101539	Home Office	Technology	Phones	Mitel MiVoice 5330e IP Phone	TEC-PH-10004165
CA-2014-36882	CA-2014-105333	Home Office	Technology	Phones	Panasonic Business Telephones KX-T7736	TEC-PH-10001468
ID-2014-26472	ID-2014-41597	Home Office	Technology	Phones	Nokia Speaker Phone, Cordless	TEC-PH-10001535
ID-2014-26474	ID-2014-41597	Home Office	Technology	Phones	Samsung Signal Booster, with Caller ID	TEC-PH-10003239

17531 rows x 35 columns

STUDENT EXERCISES:

2. sort the table by profit per unit

In [44...

```
# answer 2  
df.sort_values(by = 'Profit Per Unit', inplace = False)
```

Out[44]:

Row ID	Order ID	Segment	Category	Sub-Category	Product Name	Product ID	Country
US-2014-31980	US-2014-168116	Corporate	Technology	Machines	Cubify CubeX 3D Printer Triple Head Print	TEC-MA-10004125	United States
CA-2014-34308	CA-2014-134845	Home Office	Technology	Machines	Lexmark MX611dhe Monochrome Laser Printer	TEC-MA-10000822	United States
US-2014-36288	US-2014-122714	Corporate	Office supplies	Binders	Ibico EPK-21 Electric Binding System	OFF-BI-10001120	United States
CA-2014-36607	CA-2014-131254	Consumer	Office supplies	Binders	Fellowes PB500 Electric Punch Plastic Comb Bin...	OFF-BI-10003527	United States
CA-2014-34143	CA-2014-152093	Home Office	Office supplies	Binders	Fellowes PB500 Electric Punch Plastic Comb Bin...	OFF-BI-10003527	United States
...	...	...	...	...	...	...	...
SF-2014-48905	SF-2014-4490	Consumer	Furniture	Tables	Bevis Conference Table, Rectangular	FUR-BEV-10004805	South Africa
CA-2014-37817	CA-2014-138289	Consumer	Office supplies	Binders	GBC DocuBind P400 Electric Binding System	OFF-BI-10004995	United States
CA-2014-33920	CA-2014-127180	Home Office	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	TEC-CO-10004722	United States
CA-2014-39450	CA-2014-140151	Consumer	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	TEC-CO-10004722	United States

Row ID	Order ID	Segment	Category	Sub-Category	Product Name	Product ID	Country
CA-2014-35487	CA-2014-166709	Consumer	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	TEC-CO-10004722	United States

17531 rows x 35 columns

Table transformation, aggregation & Plotting

Internal links

- [Wide vs Long tables](#)
- [transform to a wide dataframe using .melt](#)
- [transform to a long dataframe using .pivot](#)
- [get a count of categorical variables with crosstab](#)
- [aggregate data using .groupby and and get sum using .agg](#)
- [flexible aggregation of categorical variables with .pivot_table](#)

NOTE: while this tutorial uses pandas built in plotting functions there are many better packages for plotting, the [Seaborn](#) package workes especially well with pandas dataframe and other tabular style data

```
In [45... # lets create a new dataframe for this next set of examples
# use .loc[] to select the united states as the country and .copy() to ensure
# it's a new object and not altering our base dataframe

USA_df = df.loc[df["Country"]=="United States"].copy()
```

Resetting the index

In many of the table transformations, the process of transforming the data changes the index. If we wish to retain the index row we will want to reset the index before we do these transformations so it is saved in a regular column format.

This is easily achievable with `.reset_index()`.

- Use `drop=True` if it's not necessary to maintain a column with the original indexes.

```
In [46... USA_df.reset_index(inplace=True)
USA_df
```

Out[46]:

	Row ID	Order ID	Segment	Category	Sub-Category	Product Name		
	0	CA-2014-39465	CA-2014-107174	Home Office	Furniture	Tables	Hon 5100 Series Wood Tables	10
	1	CA-2014-39466	CA-2014-107174	Home Office	Furniture	Chairs	Hon 2090 "Pillow Soft" Series Mid Back Swivel/...	F 10
	2	CA-2014-33848	CA-2014-101042	Consumer	Furniture	Furnishings	3M Polarizing Task Lamp with Clamp Arm, Light ...	F 10
	3	CA-2014-32897	CA-2014-158876	Consumer	Furniture	Furnishings	Telescoping Adjustable Floor Lamp	F 10
	4	CA-2014-39961	CA-2014-126634	Consumer	Furniture	Furnishings	Flat Face Poster Frame	F 10
	...	...	...	...	...	...	...	...
	3314	US-2014-33227	US-2014-101539	Home Office	Technology	Phones	Mitel MiVoice 5330e IP Phone	T 10
	3315	CA-2014-36882	CA-2014-105333	Home Office	Technology	Phones	Panasonic Business Telephones KX-T7736	T 10
	3316	CA-2014-40289	CA-2014-159100	Corporate	Technology	Accessories	Memorex Mini Travel Drive 8 GB USB 2.0 Flash D...	T 10
	3317	CA-2014-36007	CA-2014-138149	Consumer	Technology	Accessories	Enermax Briskie RF Wireless Keyboard and Mouse...	T 10
	3318	US-2014-36486	US-2014-136679	Consumer	Technology	Accessories	V7 USB Numeric Keypad	T 10

3319 rows x 36 columns

Wide vs Long dataframes

 No description has been provided for this image

Wide

- methods for creating: `df.pivot()` and `pd.pivot_table(df)` can reshape a long dataframe into a wide dataframe

Long

- methods for creating: `pd.melt(df)` can reshape a wide dataframe into a long dataframe

create a long dataframe with `.melt()`

`.melt` gathers columns into rows.

It can be especially effective if you have several columns related columns containing many nans that would be better represented (and reduce the number of nans) if gathered into rows

 No description has been provided for this image

```
df_melt = pd.melt(df, id_vars = [ columns ], value_vars= [
columns ], var_name = string, value_name = string)
```

- `id_vars` : Column(s) to use as identifier variables
 - columns you want to retain/ keep intact)
- `value_vars` : Column(s) to unpivot. If not specified, uses all columns that are not set as `id_vars`.
 - columns to transform
- `var_name` : Name to use for the 'variable' column. If None it uses `frame.columns.name` or 'variable'.
 - what are the columns you want to transform (are they cities? dates? something else?)
- `value_name` : Name to use for the 'value' column.
 - what are the values of the columns you are going to transform (cost? population? temperature? etc?)

 No description has been provided for this image

In our next example we want to transform our data from wide to long so we can concentrate NaNs to a single column to deal with removing them

 No description has been provided for this image

```
In [47... # You may have noticed that there are many missing values in the columns
# that contain each month, ie "1/1/2014".
# We will gather these values from columns to rows, creating two new columns
# the variables are the dates and the values represent sales

# the column labels (var_name) all represent order date
# the values(value_name) represent sales

# try to keep your formatting legible by doing like so:
columns_to_keep = ['Row ID', 'Order ID', 'Segment', 'Category', 'Sub-Category',
                  'Product ID', 'Country', 'Market', 'Region', 'City', 'State',
                  'Quantity', 'Discount', 'Profit', 'Customer ID', 'Customer Name',
                  'Order Priority', 'Postal Code', 'Ship Mode', 'Shipping Weight']

columns_to_transform = ['10/1/2014', '7/1/2014', '11/1/2014', '9/1/2014',
                       '12/1/2014', '8/1/2014', '5/1/2014', '3/1/2014', '4/1/2014',
                       '2/1/2014', '6/1/2014']

USA_long = pd.melt(USA_df,
                  id_vars = columns_to_keep,
                  value_vars = columns_to_transform,
                  var_name = "Order Date",
                  value_name = "Sales")
```

```
In [48... # We can now see the new Dataframe, USA_long
USA_long
```

Out[48]:

	Row ID	Order ID	Segment	Category	Sub-Category	Product Name
0	CA-2014-39465	CA-2014-107174	Home Office	Furniture	Tables	Hon 5100 Series Wood Tables
1	CA-2014-39466	CA-2014-107174	Home Office	Furniture	Chairs	Hon 2090 "Pillow Soft" Series Mid Back Swivel/...
2	CA-2014-33848	CA-2014-101042	Consumer	Furniture	Furnishings	3M Polarizing Task Lamp with Clamp Arm, Light ...
3	CA-2014-32897	CA-2014-158876	Consumer	Furniture	Furnishings	Telescoping Adjustable Floor Lamp
4	CA-2014-39961	CA-2014-126634	Consumer	Furniture	Furnishings	Flat Face Poster Frame
...	...	...	...	...	...	...
39823	US-2014-33227	US-2014-101539	Home Office	Technology	Phones	Mitel MiVoice 5330e IP Phone
39824	CA-2014-36882	CA-2014-105333	Home Office	Technology	Phones	Panasonic Business Telephones KX-T7736
39825	CA-2014-40289	CA-2014-159100	Corporate	Technology	Accessories	Memorex Mini Travel Drive 8 GB USB 2.0 Flash D...
39826	CA-2014-36007	CA-2014-138149	Consumer	Technology	Accessories	Enermax Briskie RF Wireless Keyboard and Mouse...
39827	US-2014-36486	US-2014-136679	Consumer	Technology	Accessories	V7 USB Numeric Keypad

39828 rows x 23 columns

```
In [49... # drop rows with empty values in the 'Sales' column
usa_sales_long = USA_long.dropna(subset=['Sales'], inplace = False).copy()
```

```
In [50... # Now that we have our long dataframe
# Lets create a new column cost of goods "COGS"
# we can calculate the cost of goods by subtracting the profits from the
usa_sales_long['COGS'] = USA_long['Sales'] - USA_long['Profit']
```

create a wide dataframe with .pivot()

`.pivot` allows us to transform rows into columns

 No description has been provided for this image

example: `df.pivot(columns = [ column(s) ])`

Parameters:

- `columns` : Column to use to make new frame's columns.
- `index` (optional): Column to use to make new frame's index. If None, uses existing index.
- `values` (optional): Column(s) to use for populating new frame's values. If not specified, all remaining columns will be used and the result will have hierarchically indexed columns.

 No description has been provided for this image

```
In [51... # get just the 'state' and 'Category' columns and turn the 'Category' col
USA_COGS_wide = usa_sales_long.pivot(index = ['Row ID'],
                                     columns='Order Date',
                                     values = 'Sales')

# view the resulting dataframe.
USA_COGS_wide
```

Out[51]:	Order Date	1/1/2014	10/1/2014	11/1/2014	12/1/2014	2/1/2014	3/1/2014	4/1/2014	5/1/2014
	Row ID								
	CA- 2014- 31309	NaN	NaN	NaN	NaN	NaN	NaN	15.552	
	CA- 2014- 31331	NaN	29.472	NaN	NaN	NaN	NaN	NaN	
	CA- 2014- 31338	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
	CA- 2014- 31340	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
	CA- 2014- 31368	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
	...	...	...	...	...	...	...	...	
	US- 2014- 41126	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
	US- 2014- 41173	NaN	NaN	NaN	NaN	NaN	NaN	848.544	
	US- 2014- 41174	NaN	NaN	NaN	NaN	NaN	NaN	8.700	
	US- 2014- 41175	NaN	NaN	NaN	NaN	NaN	NaN	122.382	
	US- 2014- 41219	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1'

3319 rows × 12 columns

STUDENT EXERCISES:

1. what are some types of plots or analysis that would be facilitated by having a wide form dataframe?
2. what about a long form dataframe?

Aggregate and Get Frequency Counts

`.crosstab()` allows you to aggregate by category and get frequency counts.

Cross-tabulations or contingency tables are tables used to describe relationships between two categorical variables. The table displays the frequency distribution of the variables. This is a very easy built in function to use, but the downside is that it is not flexible. It only offers frequency counts.

```
x_tab = pd.crosstab(index = df[categorical column],
columns=df[categorical column])
```

- `index` Values to group by in rows
- `columns` Values to group by in columns

```
In [52... # In this example we want to be able to count number of orders
# by order priority and shipping mode

orders_xtab = pd.crosstab(USA_long['Order Priority'],
                           columns = USA_long['Ship Mode'])

# view the results
orders_xtab
```

Out [52]: **Ship Mode** **First Class** **Same Day** **Second Class** **Standard Class**

Order Priority

Critical	1368	516	732	0
High	3336	1164	3288	5244
Low	0	0	0	1956
Medium	2172	552	3864	15636

aggregate data using `.pivot_table()`

`.pivot_table()` allows us to construct quick aggregate tables using categorical variables. This is slightly more complicated than `crosstab` but it offers more flexibility in aggregator functions.

```
example: pd.pivot_table(df, values = [ ], index = [ ], columns =
[ ], aggfunc = )
```

- `values` : The column to aggregate (if blank, will aggregate all numerical values)

- `index` : The column or columns to group data by. A single column can be a string, while multiple columns should be a list of strings
- `columns` : The column or columns to group data by. A single column can be a string, while multiple columns should be a list of strings
- `aggfunc` : the type of aggregator function to use, you can use one or many
 - examples: 'count', 'Mean': np.mean, 'Sigma': np.std

```
In [53... # To see how this works lets make something similar to our crosstab with

# Lets use Order ID as our values because it's unique
# set our index to "Order Priority"
# use "Ship Mode" as our columns
# and lets count as our aggregate function to count how many
# Order IDs there are for each order priority and shipping mode

pd.pivot_table(USA_long,
                values = ['Order ID'],
                index = ['Order Priority'],
                columns = ['Ship Mode'],
                aggfunc = ['count'])
```

Out [53]:

					count
					Order ID
Ship Mode	First Class	Same Day	Second Class	Standard Class	
Order Priority					
Critical	1368.0	516.0	732.0		NaN
High	3336.0	1164.0	3288.0		5244.0
Low	NaN	NaN	NaN		1956.0
Medium	2172.0	552.0	3864.0		15636.0

```
In [54... # Lets do the same thing but with columns that contain values rather than
# Lets use Sales as our values, and Order Priority and Segment as our ind
# we can also get sum and mean at the same time!

pd.pivot_table(USA_long,
                values = ['Sales'],
                index = ['Order Priority'],
                columns = ['Segment'],
                aggfunc = [np.sum, np.mean])
```

Out [54]:

Segment	sum			m		
	Sales			Sa		
Order	Consumer	Corporate	Home	Consumer	Corporate	Ho
Priority	Office					Of
Critical	15523.3562	13242.7100	12487.2920	147.841488	217.093607	240.140
High	103756.7947	81056.0856	68417.2380	185.944077	263.169109	310.987
Low	12470.4220	12673.7510	7726.4540	148.457405	264.036479	249.240
Medium	200722.4870	135038.6858	70831.7469	216.762945	239.006524	196.209

grouping data by category with .groupby()

.groupby() allows you to group a dataframe using a mapper or by a Series of columns. This can be used to group large amounts of data and compute operations on these groups.

```
grouped_object = df.groupby(by=[columns])
```



No description has been provided for this image

Groupby objects are not viewable the same way regular dataframes are. When you create a groupby object in a jupyter notebook the output will look something like so:



No description has been provided for this image

To access and view individual dataframes/groups from the groupby object you can use **.get_group()**

- `grouped_object.get_group('group')`



No description has been provided for this image

```
In [55... # Lets practice creating a groupby object.

# first lets subset usa sales long and group it by region
region_agg = usa_sales_long[['Region', 'Category', 'COGS']].groupby('Region')

# then we can view the dataframe of one of our region groups
region_agg.get_group('Central')
```

Out [55]:

	Region	Category	COGS
55	Central	Furniture	194.2044
98	Central	Furniture	420.1578
122	Central	Furniture	109.2978
123	Central	Furniture	1083.0670
425	Central	Furniture	286.7226
...	...	...	...
39473	Central	Technology	68.0157
39728	Central	Technology	78.1868
39737	Central	Technology	41.9860
39800	Central	Technology	352.9578
39801	Central	Technology	407.9400

778 rows × 3 columns

Aggregating and Statistics with groupby

You can run analytics or further aggregations on each of the groups in your groupby object.

A groupby aggregation operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups. You can use many of the descriptive statistics functions we talked about early in the notebook or `.agg()` will also compute aggregation on the grouped object. Example aggregators: min, max, sum, mean, median etc.

- `grouped_object.max()`
- `grouped_object.agg([aggregator])`

Groupby "pipeline":



No description has been provided for this image

```
In [56... # This last is gonna be slightly more complicated!

# Now lets subset the dataframe to just "Region" and "COGS"
COGS_df = usa_sales_long[['Region', 'COGS']]

# Groupby Region and get the sum for each region
region_cogs = COGS_df.groupby(by = 'Region').agg(['sum'])
```



```
# Lets view the output dataframe
region_cogs
```

Out [56]:

COGS	
sum	
Region	
Central	139547.2840
East	180044.0698
South	114116.2564
West	206731.8999

Adding aggregated data back to original dataframe

There may be cases where you don't want a whole new dataframe but would just like to add a column of aggregated data back to your original dataframe. In that case the `.transform()` function may be useful to you.

```
df('new_column') = df.groupby(by='category_column')
[['data_column']].transform(statistical_function)
```

Groupby "pipeline":



No description has been provided for this image

In [57...

```
# Final Example!
```

```
# Lets do a similar thing as above (getting cost of goods by reion) but p
usa_sales_long['regional_COGS'] = usa_sales_long.groupby(by = 'Region')[[

# Lets view the output dataframe (subset it so we can see the relevant co
usa_sales_long[["Region", "COGS", "regional_COGS"]]
```

Out [57]:

	Region	COGS	regional_COGS
18	East	160.1168	180044.0698
19	East	20.3000	180044.0698
21	East	254.5008	180044.0698
22	East	765.4752	180044.0698
38	West	177.7113	206731.8999
...	...	...	...
39801	Central	407.9400	139547.2840
39811	East	167.9790	180044.0698
39812	South	2015.9424	114116.2564
39813	South	212.9787	114116.2564
39826	West	39.8784	206731.8999

3319 rows × 3 columns

STUDENT EXERCISES:

- 1) What State has the highest Profit in the US?
- 2) Which Customer had the 3rd highest total Sales?

HINT: use the index to easily access sorted data by location

In [58... `# Answer 1`
`# aggregate the correct data & sort it`
`profit_by_state = USA_long[['State', 'Profit']].groupby('State').agg(['sum'])`
`profit_by_state.sort_values(by=['Profit', 'sum'], ascending = False, inplace=True)`
`# print state with highest profit`
`print("Most profitable state: " + profit_by_state.index[0])`

Most profitable state: California

In [59... `# Answer 2`
`# same process as above`
`sales_by_cust = USA_long[['Customer Name', 'Sales']].groupby('Customer Name').agg(['sum'])`
`sales_by_cust.sort_values(by=['Sales', 'sum'], ascending = False, inplace=True)`
`# get the 3rd row`
`print("Customer with 3rd highest total sales: " + sales_by_cust.index[2])`

Customer with 3rd highest total sales: Hunter Lopez

Creating, Joining and Saving dataframes

Internal links

Creating DataFrames from Scratch:

- [from an array](#)
- [from a lists](#)
- [from a dictionary](#)

Combining DataFrames

- [append tables with the same structure using .concat](#)
- [resetting an index](#)
- [merging dataFrames](#)
- [left and right merges](#)
- [inner and outer merges](#)

Saving DataFrames

- [save to excel](#)
- [save to .csv](#)

Creating a DataFrame

You can create dataframes from scratch by using `pd.DataFrame()`

Because dataFrames are tabular, your data will need column names. This usually means that creating a dataframe from scratch will require some combination of using arrays, lists or dictionaries.

Create a dataframe from a numpy array

When creating a dataframe from an array it's important that you know what each dimension of your array represents. Lists and dictionaries are a little more self-documenting in that way, but using arrays can be very efficient.

You must always provide the correct number of column labels to match your array dimensions

- `pd.DataFrame(data_array, columns=['column_1', 'column_2'...])`

Note: for this example we will have to create a numpy array (using `numpy.random.rand`), however in the real world you would already have your array that you'd like to turn into a dataframe.

```
In [60... # first create the array using numpy random.rand(rows,columns)
data_array = np.random.rand(25,3)
data_array
```

```
Out[60]: array([[0.94089409, 0.99437638, 0.34825674],
 [0.39350222, 0.42914181, 0.81679549],
 [0.2926979 , 0.10972205, 0.20551207],
 [0.0241317 , 0.05778678, 0.57032195],
 [0.44972355, 0.11096064, 0.5995828 ],
 [0.01688936, 0.30851396, 0.57633888],
 [0.65591498, 0.39894668, 0.30983866],
 [0.02852931, 0.85904886, 0.5649013 ],
 [0.10784915, 0.7774175 , 0.62808401],
 [0.07979877, 0.17097564, 0.19007856],
 [0.09044871, 0.61753004, 0.92740705],
 [0.67652248, 0.41647013, 0.91308224],
 [0.98830006, 0.93536641, 0.72202025],
 [0.82361695, 0.3799063 , 0.36898376],
 [0.34014229, 0.57296547, 0.75890696],
 [0.3836825 , 0.76483189, 0.39497536],
 [0.31895209, 0.13012006, 0.41675199],
 [0.33417315, 0.40122817, 0.13454778],
 [0.47056778, 0.552027 , 0.57372412],
 [0.4942463 , 0.9396013 , 0.76922181],
 [0.48182307, 0.46447142, 0.78025027],
 [0.15837629, 0.34403394, 0.04934588],
 [0.24228297, 0.53458904, 0.22867808],
 [0.49173189, 0.36208395, 0.30770382],
 [0.42775893, 0.83762424, 0.433517  ]])
```

```
In [61... # create the dataframe, assigning data first and then the columns
# our array is 25 rows x 3 columns so we must provide 3 column labels
array_df = pd.DataFrame(data_array, columns=['column_1','column_2','column_3'])

#view the dataframe
array_df
```

Out[61]:

	column_1	column_2	column_3
0	0.940894	0.994376	0.348257
1	0.393502	0.429142	0.816795
2	0.292698	0.109722	0.205512
3	0.024132	0.057787	0.570322
4	0.449724	0.110961	0.599583
5	0.016889	0.308514	0.576339
6	0.655915	0.398947	0.309839
7	0.028529	0.859049	0.564901
8	0.107849	0.777418	0.628084
9	0.079799	0.170976	0.190079
10	0.090449	0.617530	0.927407
11	0.676522	0.416470	0.913082
12	0.988300	0.935366	0.722020
13	0.823617	0.379906	0.368984
14	0.340142	0.572965	0.758907
15	0.383682	0.764832	0.394975
16	0.318952	0.130120	0.416752
17	0.334173	0.401228	0.134548
18	0.470568	0.552027	0.573724
19	0.494246	0.939601	0.769222
20	0.481823	0.464471	0.780250
21	0.158376	0.344034	0.049346
22	0.242283	0.534589	0.228678
23	0.491732	0.362084	0.307704
24	0.427759	0.837624	0.433517

dataframe from lists

You can easily make a dataframe from multiple lists and a dictionary.

- Each list represents a column of data.
- dictionary keys represent column names
- dictionary values are the lists that contain all the data/rows

Note: all lists must be the same length!

For the lists and dictionaries examples we will make a dataframe of adoptable pets!

```
In [62... # first make the lists, we will have 4 entries for each list
name = ['Oreo', 'Squid', 'Mrs Noris', 'Dazzler']
age_months = [5, 18, 12, 9]

# then assign the lists to appropriate column names using dictionary keys
dict_of_lists = {'name': name,
                 'age_months': age_months}

# finally create a dataframe using the dictionary
dict_df1 = pd.DataFrame(dict_of_lists)
dict_df1
```

Out[62]:

	name	age_months
0	Oreo	5
1	Squid	18
2	Mrs Noris	12
3	Dazzler	9

```
In [63... # Another way to make a dataframe using lists and dictionaries is to do the
# assignment within the dataframe creation call. This just cuts out a mid

names = ['Cheddar', 'Zuko']
age_months = [7, 21]

dict_df2 = pd.DataFrame({'name': names,
                        'age_months': age_months})
dict_df2
```

Out[63]:

	name	age_months
0	Cheddar	7
1	Zuko	21

dataframe from list of dictionaries

You can also create a single list that contains multiple dictionaries.

- each dictionary represents a row
- each key represents a column.

note: you must use the exact same keys in all dictionaries using this method.

```
In [64... list_of_dicts = [  
    {'name': 'Oreo',      'species': 'dog'},  
    {'name': 'Dazzler',  'species': 'cat'},  
    {'name': 'Templeton', 'species': 'rat'},  
]  
  
list_df = pd.DataFrame(list_of_dicts)  
list_df
```

```
Out[64]:
```

	name	species
0	Oreo	dog
1	Dazzler	cat
2	Templeton	rat

Append dataframes using concat

you can append one dataframe to another if they have the same structure(i.e. same columns OR same indexes) using `.concat()`

concatenate rows: `pd.concat([df1, df2])`



No description has been provided for this image

concatenate columns: `pd.concat([df1, df2], axis=1)`



No description has been provided for this image

```
In [65... # dict_df1 and dict_df2 have the same structure without redundancy  
# so they're good candidates for concatenation.  
  
pd.concat([dict_df1, dict_df2])
```

```
Out[65]:
```

	name	age_months
0	Oreo	5
1	Squid	18
2	Mrs Noris	12
3	Dazzler	9
0	Cheddar	7
1	Zuko	21

In [66... *# Lets try concatenating again but resetting the index while we do so*

```
pd.concat([dict_df1, dict_df2]).reset_index(drop=True)
```

Out[66]:

	name	age_months
0	Oreo	5
1	Squid	18
2	Mrs Noris	12
3	Dazzler	9
4	Cheddar	7
5	Zuko	21

Merging DataFrames

Needing to merge or join dataframes is a very common occurrence in data analysis. Frequently one table contains a specific type of data while another table contains some other data, and you wish to combine that information.

There are several different ways to merge DataFrames. The most common way is using `.merge()`

Important function arguments:

- `how` determines the way the dataframes are merged. There are 4 basic options:
 - `'left' & 'right'`
 - `'inner' (default) & 'outer'`
- `on` argument identifies which column(s) are common across both dataframes and should be used as the index to merge the dataframes on
 - in the example images below `'x1'` is the common column that would be used
 - `on = 'x1'`

Left and Right merges

For `'left'` and `'right'` merges, the order that you list the dataframes in the `.merge()` function matters. The first dataframe listed is the `'left'` dataframe and the second is the `'right'` dataframe.

- left merge:
 - Joins matching rows from df_right to df_left, retaining all the information from the df_left.
 - `left_merge_df = pd.merge(df_left, df_right, how='left', on='x1')`

 No description has been provided for this image

- right merge example:
 - Joins matching rows from df_left to df_right, retaining all the information from the df_right
 - `right_merge_df = pd.merge(left_df, right_df, how='right', on='x1')`

 No description has been provided for this image

```
In [67... # Lets do a LEFT join
# dict_df1 which contains information on animal 'name' and 'age_months'
# list_df contains 'name' and 'species'

pd.merge(dict_df1, list_df, how = 'left', on = 'name')
```

Out[67]:

	name	age_months	species
0	Oreo	5	dog
1	Squid	18	NaN
2	Mrs Noris	12	NaN
3	Dazzler	9	cat

```
In [68... # Try a right join with the same dataframes!

pd.merge(dict_df1, list_df, how = 'right', on = 'name')
```

Out[68]:

	name	age_months	species
0	Oreo	5.0	dog
1	Dazzler	9.0	cat
2	Templeton	NaN	rat

Inner and Outer Merges

For 'inner' and 'outer' merges, the order of the dataframes does not matter. Instead the inner and outer merges are based on what is common between both dataframes.

- inner merge:
 - Join data and retains only rows found in both dataframes.
 - `inner_merge_df = pd.merge(df1, df2, how='inner', on='x1')`

 No description has been provided for this image

- outer merge:
 - Joins dataframes and retains all values, all rows.
 - `outer_merge_df = pd.merge(df1, df2, how='outer', on='x1')`

 No description has been provided for this image

```
In [69... # Lets try an outer merge with those same dataframes

pd.merge(dict_df1, list_df, how = 'outer', on = 'name')
```

Out [69]:

	name	age_months	species
0	Oreo	5.0	dog
1	Squid	18.0	NaN
2	Mrs Noris	12.0	NaN
3	Dazzler	9.0	cat
4	Templeton	NaN	rat

Merging with Indexes

You may have two tables that you'd like to merge where the index of one dataframe is a column in another dataframe.

 No description has been provided for this image

```
df1.merge(df2, left_on = "x3", right_index = True)
```

Parameters:

`left_on` label or list, or array-like Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

`right_on` label or list, or array-like Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

`left_index` bool, default False Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

`right_index` bool, default False Use the index from the right DataFrame as the join key. Same caveats as `left_index`.

NOTE: If you have other columns in common besides the merging column/index these will appear twice in your new dataframe!

Saving dataframes

Just like loading dataframes, there are many different formats that you can save a dataframe to. We will just show you .csv and .xls but to see a complete list please check out the [input/output documentation here](#):

- `.to_csv()`
 - `df.to_csv('df_save_name.csv')`
- `.to_excel()`
 - `df.to_excel('df_save_name.xlsx')`

by default the save functions will save to your working directory. You must specify a path if you wish to save your dataframe elsewhere.

STUDENT EXERCISES:

1. Read in population data from the csv file and show the columns (find file in: `../support_files/datasets/population_by_country_2020.csv`).
2. Perform a left merge on the original dataframe (df) and new population dataframe, using "Country" column, print columns to confirm merge
3. Return the merged dataframe from "Country" with the highest population, first 10 rows

This country population dataset was provided mostly as an example for you to trying merging two different datasets. But feel free to consider interesting questions and corresponding plots to investigate this merged data. For example, is there a difference in categories of items sold in high vs low population countries?

```
In [70... # Answer 1:
filepath = os.path.join('../support_files', 'datasets', 'population_by_co

pop_df = pd.read_csv(filepath)

print(pop_df.columns)
```

```
Index(['Country', 'Population', 'Yearly Change', 'Net Change',
       'Density (P/Km²)', 'Land Area (Km²)', 'Migrants (net)', 'Fert. Rat
e',
       'Med. Age', 'Urban Pop %', 'World Share'],
      dtype='object')
```

```
In [71... # Answer 2:
merged_df = pd.merge(df, pop_df, how = 'left', on = 'Country')
print(merged_df.columns)
```

```
Index(['Order ID', 'Segment', 'Category', 'Sub-Category', 'Product Name',
       'Product ID', 'Country', 'Market', 'Region', 'Quantity', 'Discoun
t',
       'Profit', 'Customer ID', 'Customer Name', 'Order Priority',
       'Postal Code', 'Ship Mode', 'Shipping Cost', '10/1/2014', '7/1/201
4',
       '11/1/2014', '9/1/2014', '1/1/2014', '12/1/2014', '8/1/2014',
       '5/1/2014', '3/1/2014', '4/1/2014', '2/1/2014', '6/1/2014', 'City',
       'State', 'manufacturers', 'Profit Per Unit', 'manufacturers_count',
       'Population', 'Yearly Change', 'Net Change', 'Density (P/Km²)',
       'Land Area (Km²)', 'Migrants (net)', 'Fert. Rate', 'Med. Age',
       'Urban Pop %', 'World Share'],
      dtype='object')
```

```
In [72... # Answer 3:
merged_df.sort_values(by=['Population'], ascending = False).head(10)
merged_df
```

Out[72]:

	Order ID	Segment	Category	Sub-Category	Product Name	Product ID	Count	
	0	IN-2014-75456	Consumer	Furniture	Furnishings	Rubbermaid Door Stop, Ergonomic	FUR-FU-10004064	Afghanistan
	1	IN-2014-29767	Home Office	Furniture	Bookcases	Ikea Library with Doors, Mobile	FUR-BO-10001255	Afghanistan
	2	IN-2014-29767	Home Office	Furniture	Furnishings	Rubbermaid Door Stop, Ergonomic	FUR-FU-10004064	Afghanistan
	3	IN-2014-20415	Home Office	Furniture	Bookcases	Bush Classic Bookcase, Pine	FUR-BO-10002204	Afghanistan
	4	IN-2014-47337	Corporate	Furniture	Chairs	Hon Rocking Chair, Red	FUR-CH-10003965	Afghanistan
	...	...	...	...	...	...	...	
17526	ZA-2014-9750	Corporate	Technology	Accessories	Memorex Router, USB	TEC-MEM-10002202		Zambia
17527	ZI-2014-7610	Corporate	Technology	Machines	StarTech Phone, Red	TEC-STA-10000699		Zimbabwe
17528	ZI-2014-5970	Home Office	Technology	Accessories	Belkin Router, USB	TEC-BEL-10003985		Zimbabwe
17529	ZI-2014-9550	Consumer	Technology	Machines	Konica Receipt Printer, Red	TEC-KON-10003116		Zimbabwe
17530	ZI-2014-3570	Consumer	Technology	Machines	Okidata Calculator, Red	TEC-OKI-10001433		Zimbabwe

17531 rows x 45 columns